UNIVERSITAS TEKNOLOGI BANDUNG

DIKTISAINTEK BERDAMPAK

Pertemuan 6
State Management Fundamentals

Dosen Pengampu: Nova Agustina, S.T., M.Kom.



Learning Points

1. Mahasiswa dapat menjelaskan perbedaan antara Stateless dan Stateful Widget
2. Mahasiswa dapat membuat aplikasi sederhana dengan Stateful Widget.
3. Mahasiswa dapat mengubah tampilan UI secara dinamis menggunakan setState().
4. Mahasiswa mengenal konsep dasar state management menggunakan Bloc
5. Mahasiswa dapat menerapkan Bloc sederhana untuk mengelola perubahan state di aplikasi.

UNIVERSITAS TEKNOLOGI BANDUNG

DIKTISAINTEK BERDAMPAK

Stateless & Stateful Widget

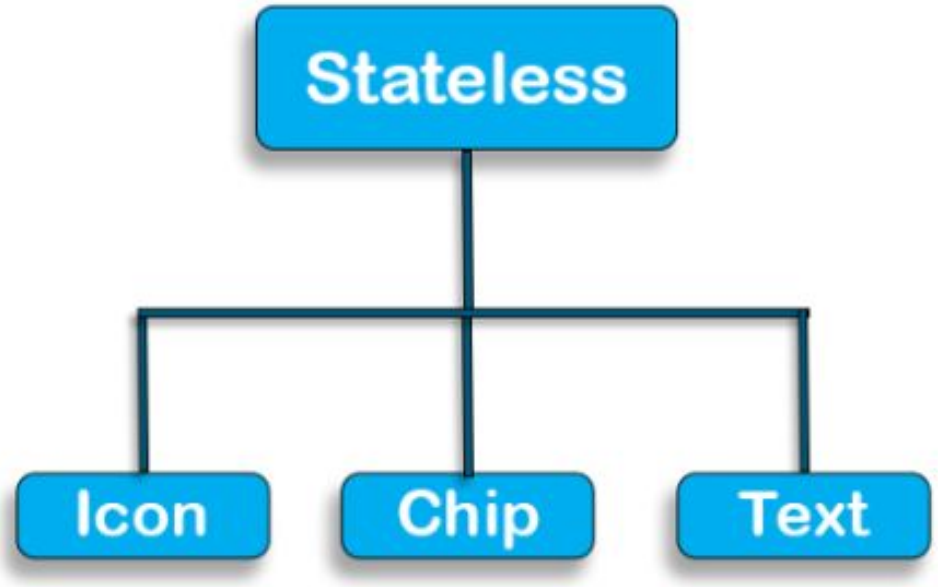
Stateless & Stateful Widget

Flutter widget dibangun dengan menggunakan modern framework yang terinspirasi dari React, dan ide utamanya itu semua komponen dalam Flutter adalah Widget. Widget menjelaskan bagaimana suatu komponen yang terlihat dan bagaimana state / kondisinya

Stateless & Stateful Widget

Stateless Widget	Stateful WIdget
Widget yang tidak memiliki state, atau kondisinya **tidak berubah-ubah**. Contoh: Ketika menampilkan gambar, text maupun widget lainnya dan tidak ada perubahan atau tetap	Widget yang memiliki state dan **dapat berubah-ubah**. Contoh : Ketika tombol di klik akan merubah komponen yang lain. **Untuk membuat StatefulWidget diperlukan membuat 2 buah class yaitu class yang mewarisi / extends ke StatefulWidget dari class extend ke state**

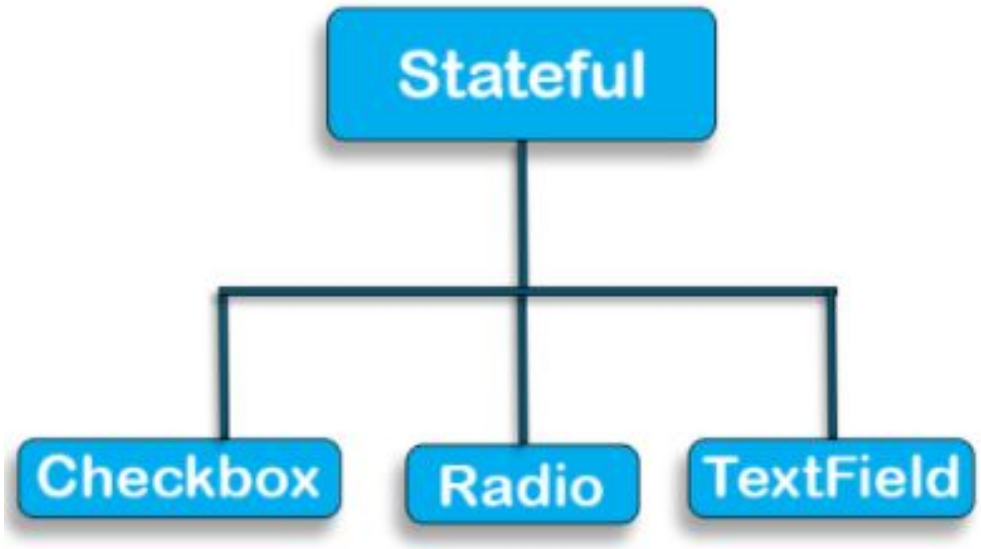
Stateless Widget



> *If you want to redraw the Stateless Widget, then you will need to create a new instance of the widget.*

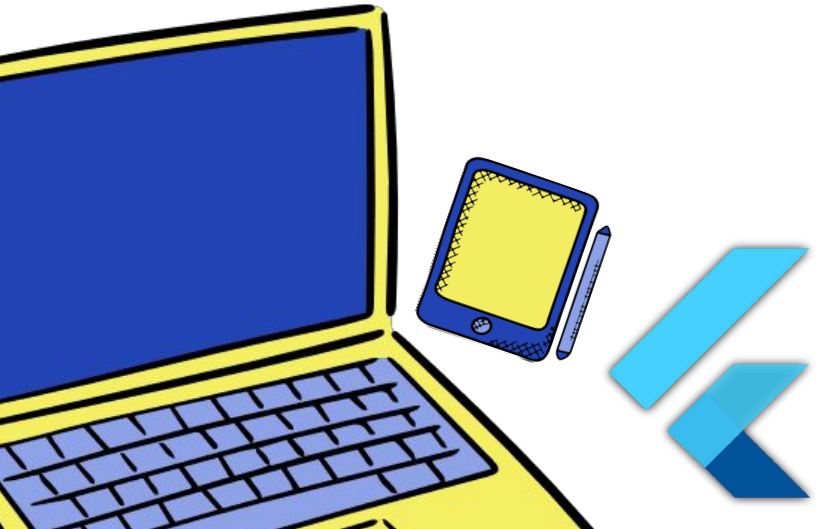
TIPS: You can quickly build a Stateless Widget in VS Code or Android Studio by using the shortcut "stl".

Stateful Widget

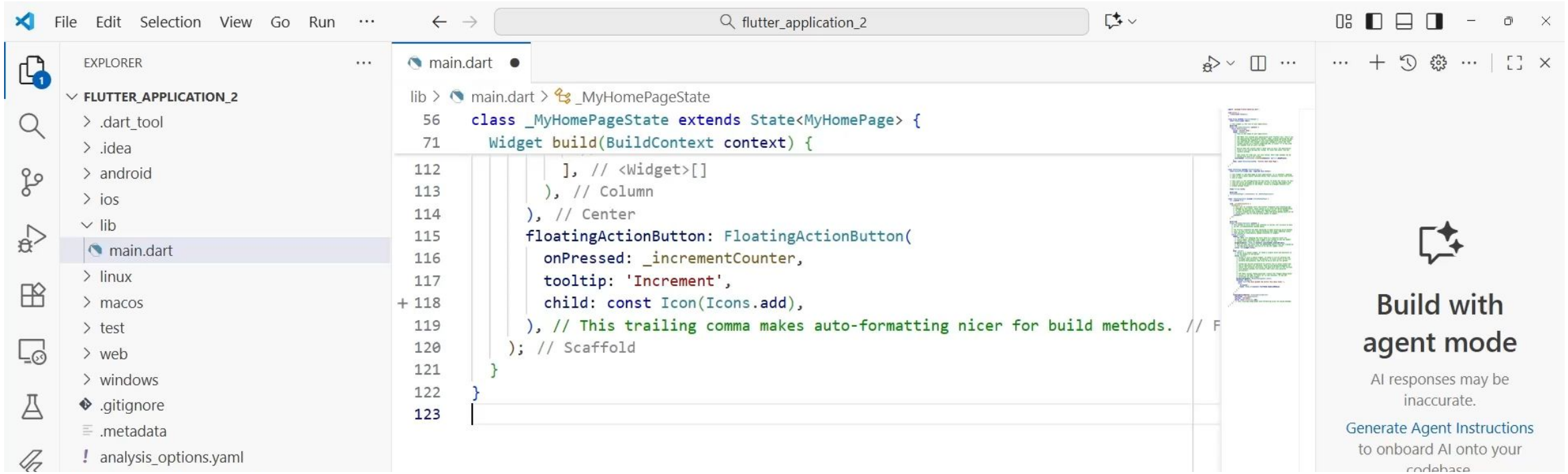


TIPS: You can quickly build a Stateful Widget in VS Code or Android Studio by using the shortcut “stf”.

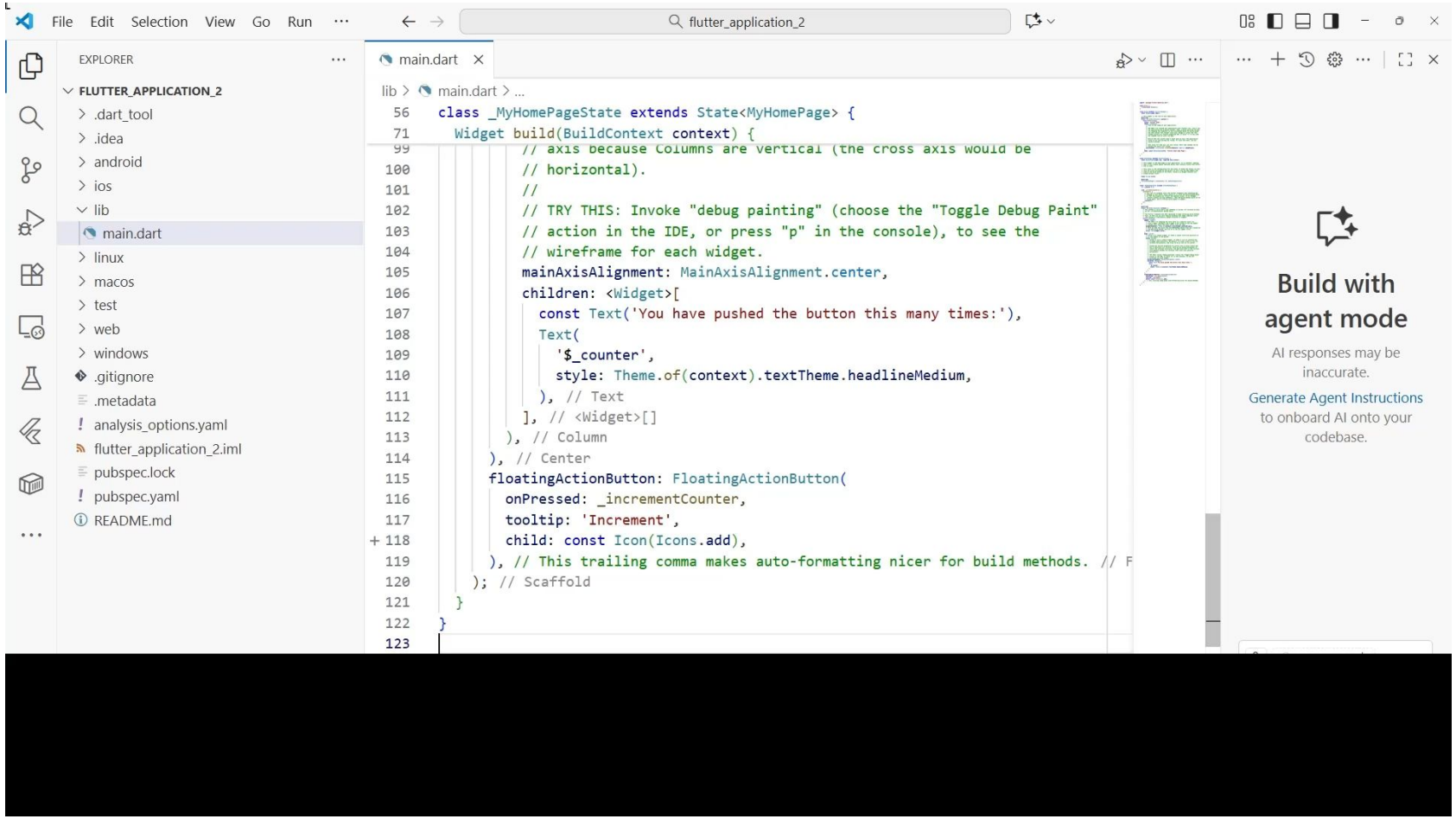
Let’s Go to VSCode...



Stateless


File Edit Selection View Go Run
flutter_application_2
EXPLORER
FLUTTER_APPLICATION_2
.dart_tool
.idea
android
ios
lib
main.dart
linux
macos
test
web
windows
.gitignore
.metadata
analysis_options.yaml
main.dart
lib > main.dart > _MyHomePageState
56 class _MyHomePageState extends State<MyHomePage> {
71 Widget build(BuildContext context) {
112], // <Widget>[]
113), // Column
114), // Center
115 floatingActionButton: FloatingActionButton(
116 onPressed: _incrementCounter,
117 tooltip: 'Increment',
118 child: const Icon(Icons.add),
119), // This trailing comma makes auto-formatting nicer for build methods.
120); // Scaffold
121 }
122 }
123
Build with agent mode
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI onto your codebase

Stateful


File Edit Selection View Go Run
flutter_application_2
EXPLORER
FLUTTER_APPLICATION_2
.dart_tool
.idea
android
ios
lib
main.dart
linux
macos
test
web
windows
.gitignore
.metadata
analysis_options.yaml
flutter_application_2.iml
pubspec.lock
pubspec.yaml
README.md
main.dart
lib > main.dart > ...
class _MyHomePageState extends State<MyHomePage> {
Widget build(BuildContext context) {
// axis because Columns are vertical (the cross axis would be
// horizontal).
//
// TRY THIS: Invoke "debug painting" (choose the "Toggle Debug Paint"
// action in the IDE, or press "p" in the console), to see the
// wireframe for each widget.
mainAxisAlignment: MainAxisAlignment.center,
children: <Widget>[
const Text('You have pushed the button this many times:'),
Text(
'$_counter',
style: Theme.of(context).textTheme.headlineMedium,
), // Text
], // <Widget>[]
), // Column
), // Center
floatingActionButton: FloatingActionButton(
onPressed: _incrementCounter,
tooltip: 'Increment',
child: const Icon(Icons.add),
), // This trailing comma makes auto-formatting nicer for build methods. // F
); // Scaffold
}
}
Build with agent mode
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI onto your codebase.

UNIVERSITAS TEKNOLOGI BANDUNG

DIKTISAINTEK BERDAMPAK

SetState

lib > main.dart > _MyAppState

```dart
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(const MyApp());
}

class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: "Latihan Statelful Widget",
      home: Scaffold(
        appBar: AppBar(title: const Text("Latihan Statelful Widget")),
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: <Widget>[],
          ), // Column
        ), // Center
      ), // Scaffold
    ); // MaterialApp
  }
}
```

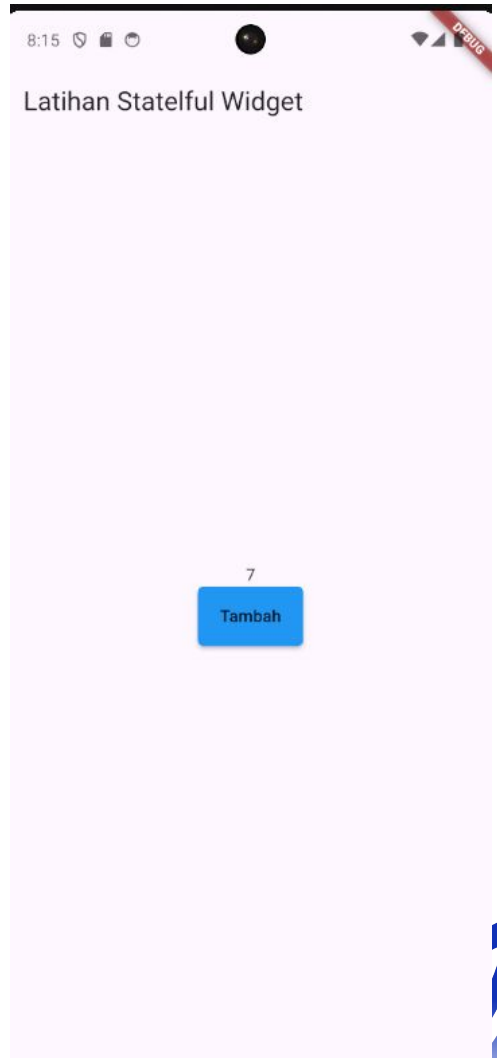
SetState

```
22          child: Column(
23            mainAxisAlignment: MainAxisAlignment.center,
24            children: <Widget>[
25              Text("0"),
26              MaterialButton(
27                onPressed: tekantombol,
28                color: Colors.blue,
29                height: 50,
30                shape: RoundedRectangleBorder(
31                  borderRadius: BorderRadius.circular(5),
32                ), // RoundedRectangleBorder
33                child: const Text("Tambah")), // MaterialButton
34            ], // <Widget>[]
35          ), // Column
```

SetState

```
14  class _MyAppState extends State<MyApp> {
15    int number = 1;
16    void tekantombol() {
17      setState(() {
18        number++;
19      });
20    }
21    @override
22    Widget build(BuildContext context) {
```

```
child: Column(
  mainAxisAlignment: MainAxisAlignment.center,
  children: <Widget>[
    Text(number.toString()),
```



UNIVERSITAS TEKNOLOGI BANDUNG

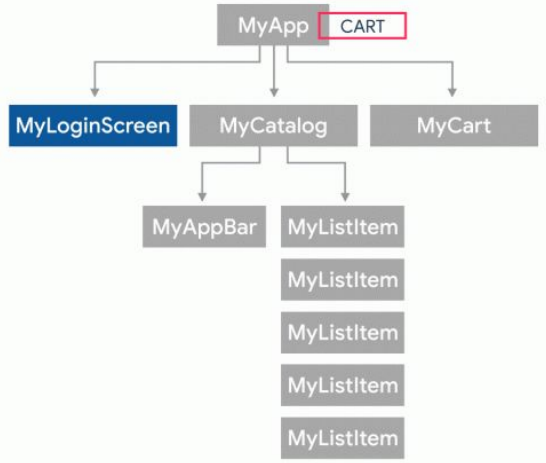
DIKTISAINTEK BERDAMPAK

Flutter BloC

State Management

State management adalah sebuah cara untuk mengatur data / state bekerja, bisa juga untuk memisahkan antara logic dan view, dimana logic tersebut juga bisa reusable.





State Management

State management bersifat deklaratif, artinya flutter membangun user interfacenya dengan merefleksikan setiap perubahan state. State ini lah yang menjadi trigger untuk me-redraw tampilan user interface. Ketika terjadi perubahan, UI akan merebuild secara otomatis dari awal.



Jenis State Management

- Provider
- Riverpod
- Redux
- BloC
- GetX

Provider

Provider merupakan state management yang paling sederhana dan mudah digunakan. Provider menyediakan sebuah teknik mengolah state yang dapat digunakan untuk memanage data didalam aplikasi. Manfaat menggunakan provider antara lain:

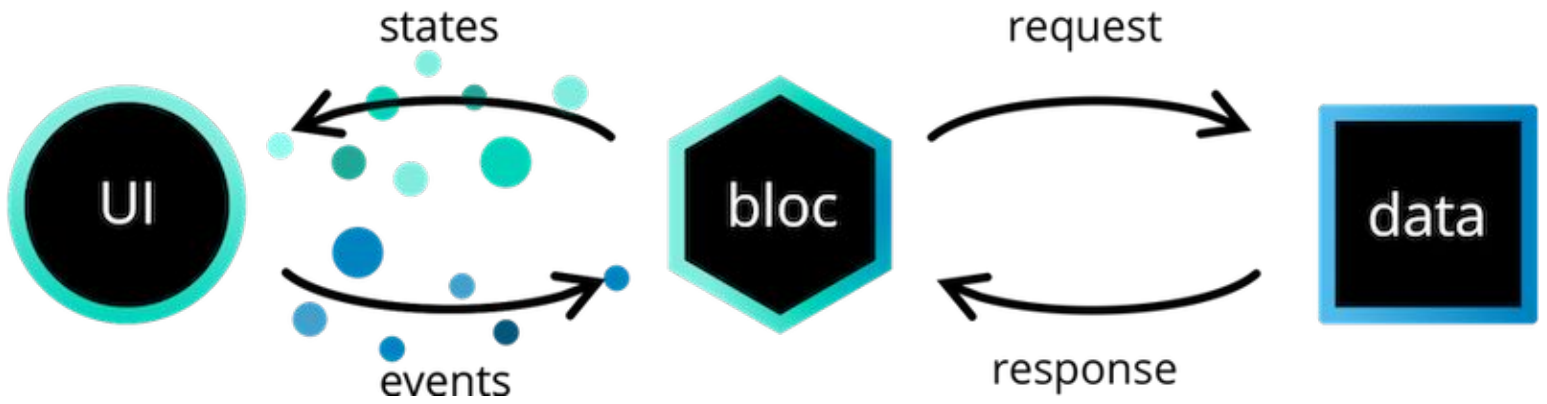
- Mengalokasikan resource menjadi lebih sederhana
- Lazy-loading (Penundaan data, jika dipakai maka data baru ditampilkan)
- Mengurangi penulisan kode berulang saat membuat kelas baru setiap saat

BloC

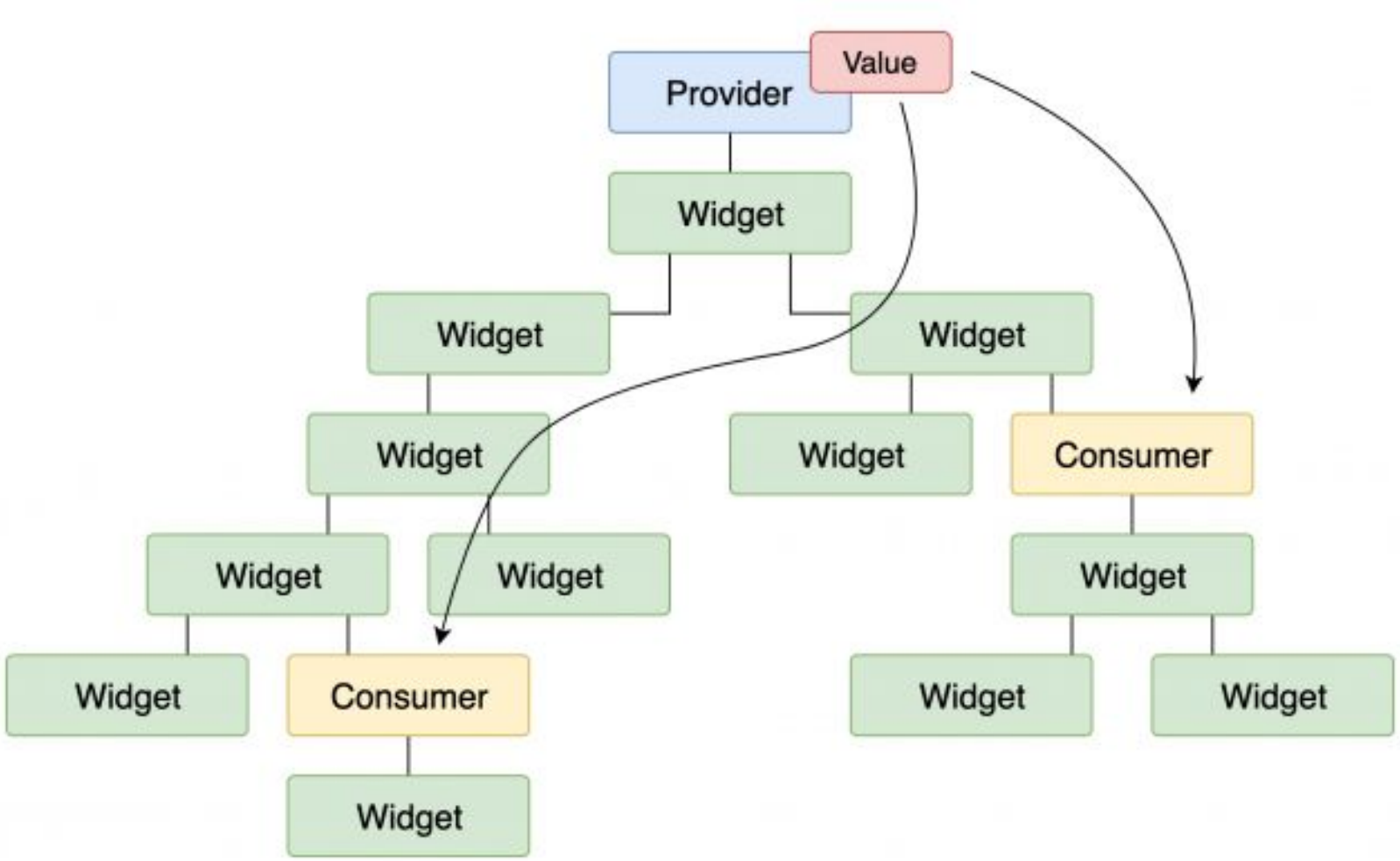
BLoC atau Business Logic Component adalah design pattern yang bertugas untuk memisahkan presentation dengan business logic. Sehingga komponen pada project terbagi menjadi presentational component, BLoC, dan backend. Pattern ini memperbolehkan developer untuk fokus dalam mengkonversikan event menjadi state.

BloC

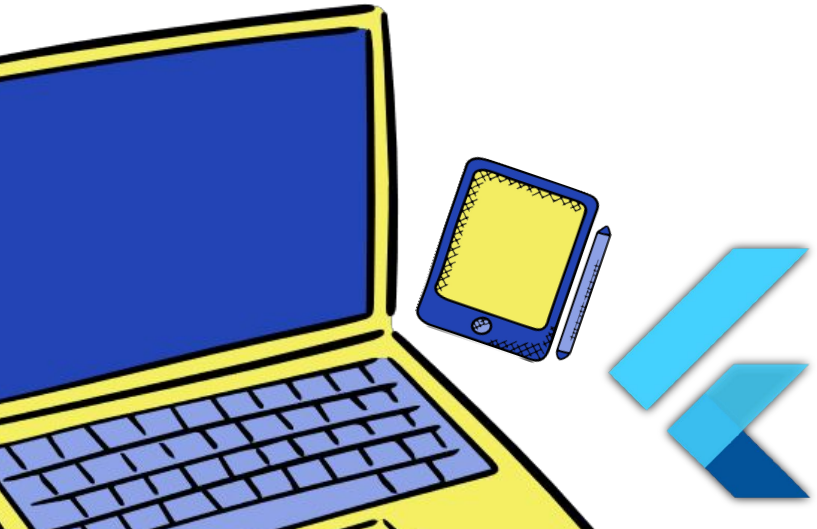
BLoC mengelola state dengan menggunakan pendekatan stream atau reactive. Secra umum, data akan bergerak dari BLOC ke UI, atau sebaliknya dalam bentuk streams.



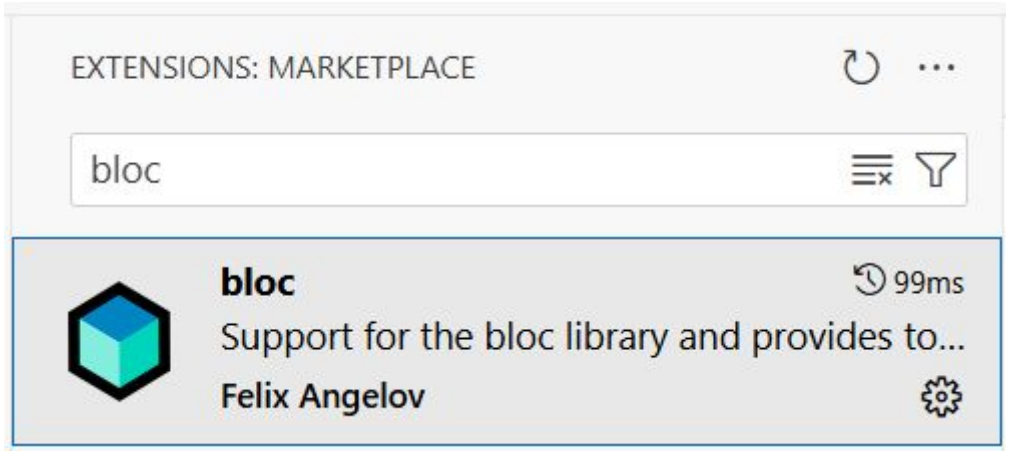
BloC

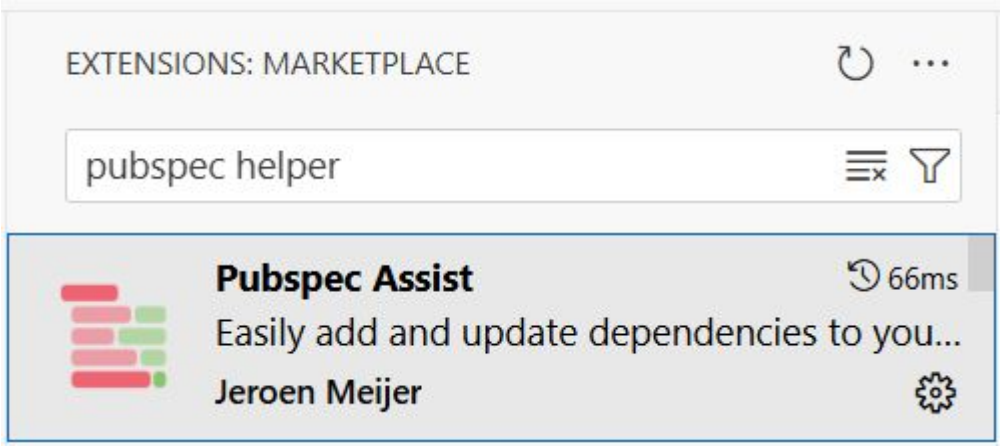

Provider
Value
Widget
Widget
Widget
Widget
Widget
Consumer
Widget
Widget
Widget
Widget
Consumer
Widget
Widget
Widget
Widget

Let’s Go to VSCode...

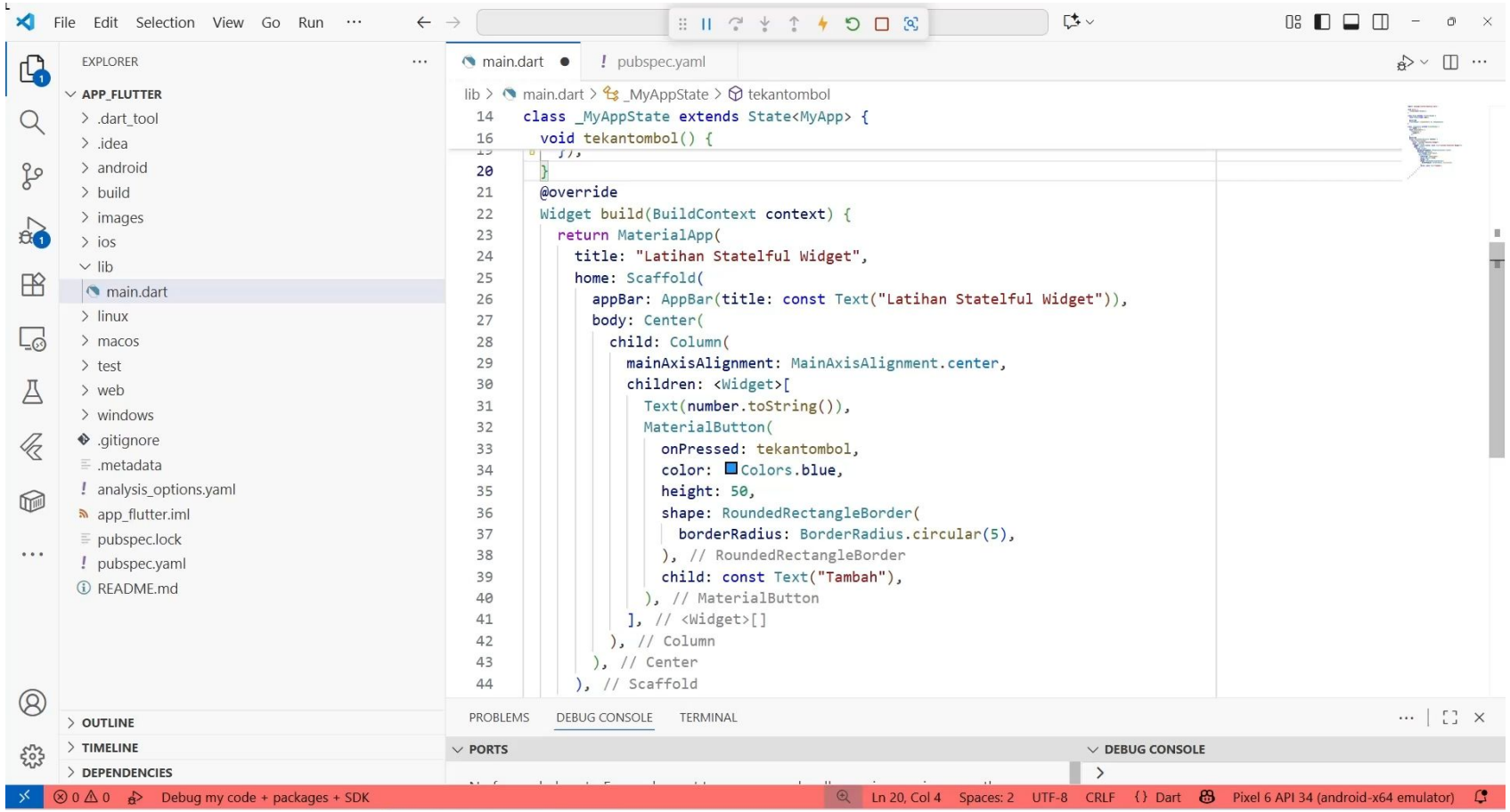


BloC


EXTENSIONS: MARKETPLACE
bloc
bloc
99ms
Support for the bloc library and provides to...
Felix Angelov

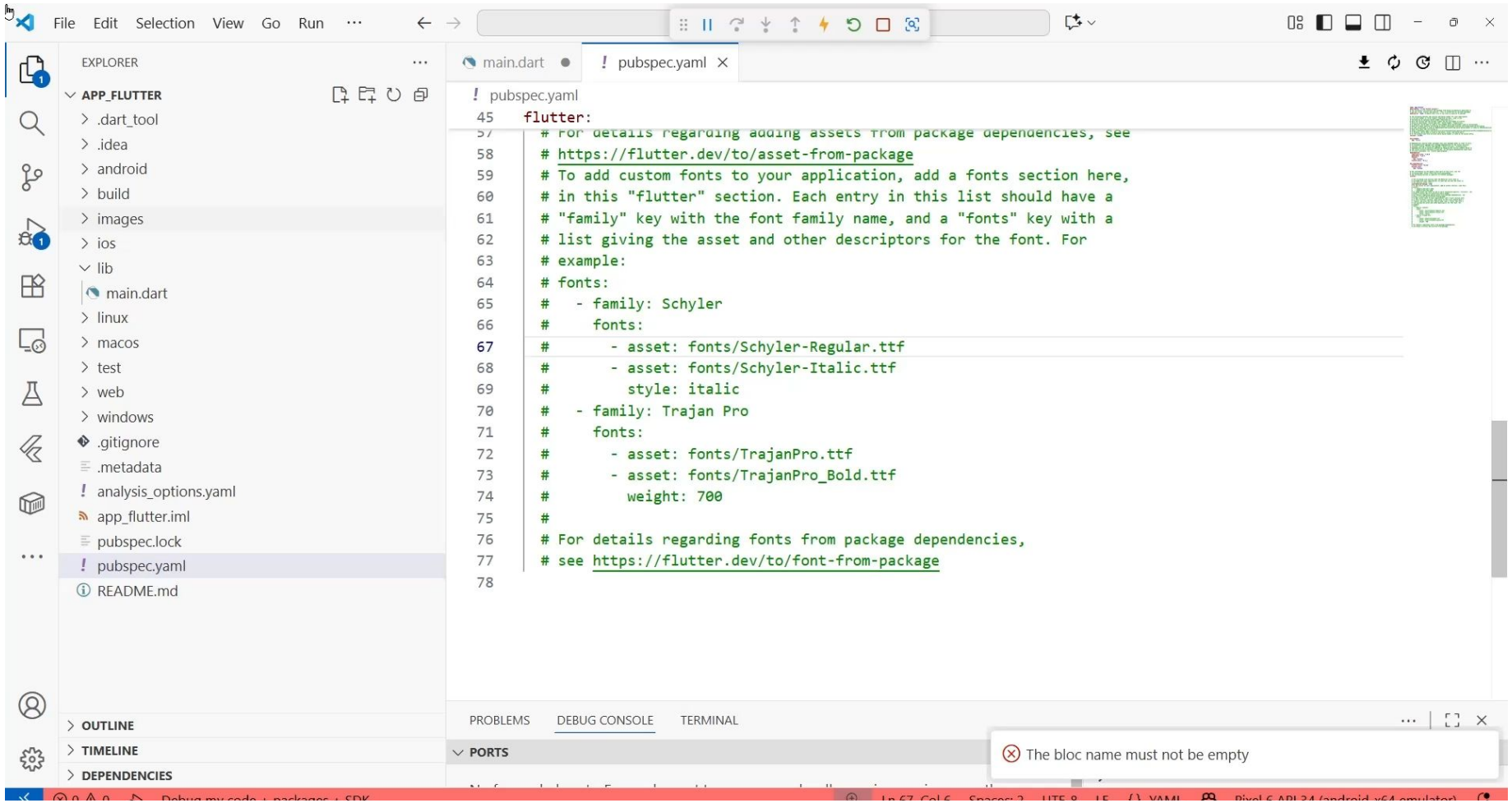

EXTENSIONS: MARKETPLACE
pubspec helper
Pubspec Assist
66ms
Easily add and update dependencies to you...
Jeroen Meijer

BloC



Tambahkan library flutter_bloc dan equatable pada project latihan_bloc

Create BloC


File Edit Selection View Go Run
EXPLORER
APP_FLUTTER
.dart_tool
.idea
android
build
images
ios
lib
main.dart
linux
macos
test
web
windows
.gitignore
.metadata
analysis_options.yaml
app_flutter.iml
pubspec.lock
pubspec.yaml
README.md
OUTLINE
TIMELINE
DEPENDENCIES
main.dart
pubspec.yaml
pubspec.yaml
45 flutter:
58 # https://flutter.dev/to/asset-from-package
59 # To add custom fonts to your application, add a fonts section here,
60 # in this "flutter" section. Each entry in this list should have a
61 # "family" key with the font family name, and a "fonts" key with a
62 # list giving the asset and other descriptors for the font. For
63 # example:
64 # fonts:
65 # - family: Schyler
66 # fonts:
67 # - asset: fonts/Schyler-Regular.ttf
68 # - asset: fonts/Schyler-Italic.ttf
69 # style: italic
70 # - family: Trajan Pro
71 # fonts:
72 # - asset: fonts/TrajanPro.ttf
73 # - asset: fonts/TrajanPro_Bold.ttf
74 # weight: 700
75 #
76 # For details regarding fonts from package dependencies,
77 # see https://flutter.dev/to/font-from-package
78
PROBLEMS
DEBUG CONSOLE
TERMINAL
PORTS
The bloc name must not be empty

BloC State

State pada Bloc berfungsi sebagai **menghasilkan status perubahan** untuk memanajemen aplikasi. Kita dapat membuat beberapa State pada 1 Event di halaman “**nama bloc**_state.dart”

BloC State

lib > bloc > latihan_bloc_satu_state.dart > ...

```dart
part of 'latihan_bloc_satu_bloc.dart';

sealed class LatihanBlocSatuState extends Equatable {
  const LatihanBlocSatuState();

  @override
  List<Object> get props => [];
}

final class LatihanBlocSatuInitial extends LatihanBlocSatuState {
  final int counter;

  const LatihanBlocSatuInitial(this.counter);

  @override
  List<Object> get props => [counter];
}
```

BloC Event

Event pada BloC menampung sebuah “Perintah” yang akan dikerjakan pada saat event tersebut dipanggil. Pada saat kita membuat BloC, kita sudah memiliki abstact class LatihanBlocSatuEvent, yang menandakan bahwa setiap kita membuat BloC, minimal harus memiliki 1 event atau kejadian. Kita dapat memperbanyak event pada BloC di file “**nama bloc_event**.dart”

BloC Event

lib > bloc > latihan_bloc_satu_event.dart > ...

```dart
part of 'latihan_bloc_satu_bloc.dart';

sealed class LatihanBlocSatuEvent extends Equatable {
  const LatihanBlocSatuEvent();

  @override
  List<Object> get props => [];
}

// Event penambah counter
class IncrementEvent extends LatihanBlocSatuEvent {}

// Event pengurang counter
class DecrementEvent extends LatihanBlocSatuEvent {}
```

BloC

Bloc berfungsi untuk menerima request dan mengeluarkan hasil ke User Interface. Dalam 1 Bloc, terdapat lebih dari satu event dan state. Ubah source pada “nama bloc_bloc.dart”

BloC

lib > bloc > latihan_bloc_satu_bloc.dart > ...

```dart
import 'package:bloc/bloc.dart';
import 'package:equatable/equatable.dart';

part 'latihan_bloc_satu_event.dart';
part 'latihan_bloc_satu_state.dart';

class LatihanBlocSatuBloc extends Bloc<LatihanBlocSatuEvent, LatihanBlocSatuState> {
  LatihanBlocSatuBloc() : super(LatihanBlocSatuInitial(0)) {
    on<LatihanBlocSatuEvent>((event, emit) {
      // TODO: implement event handler
    });
  }
}
```

BloC

latihan_bloc_satu_bloc.dart

lib > bloc > latihan_bloc_satu_bloc.dart > LatihanBlocSatuBloc > LatihanBlocSatuBloc

```dart
import 'package:bloc/bloc.dart';
import 'package:equatable/equatable.dart';

part 'latihan_bloc_satu_event.dart';
part 'latihan_bloc_satu_state.dart';

class LatihanBlocSatuBloc
    extends Bloc<LatihanBlocSatuEvent, LatihanBlocSatuState> {
  LatihanBlocSatuBloc() : super(LatihanBlocSatuInitial(0)) {
    on<IncrementEvent>((event, emit) {
      // TODO: implement event
      final current = (state as LatihanBlocSatuInitial).counter;
      emit(LatihanBlocSatuInitial(current + 1));
    });

    on<DecrementEvent>((event, emit) {
      final current = (state as LatihanBlocSatuInitial).counter;
      emit(LatihanBlocSatuInitial(current - 1));
    });
  }
}
```

Koneksi BloC ke UI

main.dart ●

lib > main.dart > MyApp

```dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: "Latihan BloC",
      home: BlocProvider(
        create: (_) => LatihanBlocSatuBloc(),
        child: const CounterPage(),
      ), // BlocProvider
    ); // MaterialApp
  }
}

class CounterPage extends StatelessWidget {
  const CounterPage({super.key});

  @override
  Widget build(BuildContext context) {
    return const Placeholder();
  }
}
```

Koneksi BloC ke UI

```
24  class CounterPage extends StatelessWidget {
25    const CounterPage({super.key});
26
27    @override
28    Widget build(BuildContext context) {
29      return Scaffold(
30        appBar: AppBar(title: const Text('Bloc Counter')),
31        body: Center(),
32        floatingActionButton: Row(
33          mainAxisAlignment: MainAxisAlignment.end,
34          children: <Widget>[],
35        ), // Row
36      ); // Scaffold
37    }
38  }
```

Koneksi BloC ke UI

```
42      floatingActionButton: Row(
43        mainAxisAlignment: MainAxisAlignment.end,
44        children: [
45          FloatingActionButton(
46            heroTag: 'minus',
47            onPressed: () {
48              context.read<LatihanBlocSatuBloc>().add(DecrementEvent());
49            },
50            child: const Icon(Icons.remove),
51          ), // FloatingActionButton
52          const SizedBox(width: 10),
53          FloatingActionButton(
54            heroTag: 'plus',
55            onPressed: () {
56              context.read<LatihanBlocSatuBloc>().add(IncrementEvent());
57            },
58            child: const Icon(Icons.add),
59          ), // FloatingActionButton
60        ],
61      ), // Row
```

Koneksi BloC ke UI

```
27  @override
28  Widget build(BuildContext context) {
29    return Scaffold(
30      appBar: AppBar(title: const Text('Counter with BLoC')),
31      body: Center(
32        child: BlocBuilder<LatihanBloc1Bloc, LatihanBloc1State>(
33          builder: (context, state) {
34            final counter = (state as LatihanBloc1Initial).counter;
35            return Text('$counter', style: const TextStyle(fontSize: 24));
36          },
37        ), // BlocBuilder
38      ), // Center
```

UNIVERSITAS TEKNOLOGI BANDUNG

DIKTISAINTEK BERDAMPAK

TERIMA KASIH

Kontak: **081320329095**

Email: **nova@utb-univ.ac.id**

